

Chapitre 4 : Retour Critique : LLMs Embarqués sur Smartphone

4.1 Limites actuelles des modèles on-device

4.1.1 Raisonnement et logique formelle

Les modèles embarqués (<4B paramètres) présentent des limitations structurelles sur le raisonnement :

Tâche	Gemma 2 2B	Phi-3 Mini (3,8B)	Gemini Nano-2	GPT-4o (référence cloud)
GSM8K (arithmétique)	46 %	82,5 %	~72 %	92 %
ARC-Challenge (logique)	51 %	61 %	~65 %	96 %
MMLU (connaissances)	52 %	68,8 %	79,6 %	88 %
Raisonnement multi-étapes	Faible	Moyen	Moyen	Excellent

Tableau 4.1 : Capacités de raisonnement par modèle, comparées à une référence cloud (GPT-4o)

Conclusion : les modèles on-device sont viables pour les tâches simples (Q/R factuelles, résumé, classification) mais insuffisants pour les tâches nécessitant un raisonnement enchaîné complexe (déduction logique, mathématiques avancées, planification longue).

4.1.2 Fenêtre de contexte limitée

La contrainte RAM impose des contextes courts :

Modèle	Contexte max théorique	Contexte pratique sur mobile
Gemma 2 2B Q4	8 192 tokens	2 048 tokens (3-4 Go RAM)
LLaMA 3.2 3B Q4	128 000 tokens	2 048-4 096 tokens
Gemini Nano-2	Non publié	~2 048 tokens (AICore)

Tableau 4.2 : Fenêtre de contexte théorique et pratique par modèle

Au-delà de 2 048 tokens de contexte, la consommation RAM devient critique sur les appareils avec 6-8 Go disponibles, provoquant des SIGKILL (OOM killer Android).

Impact pratique : impossibilité d'analyser des documents longs (articles académiques, contrats), de maintenir des conversations très longues, ou de faire de la RAG (Retrieval-Augmented Generation) avec de grands chunks.

4.1.3 Stabilité et hallucinations

Les modèles compressés (INT4) présentent une légère augmentation des hallucinations par rapport à leurs versions FP16 :

- Environ 3-5 % de dégradation sur les benchmarks de fidélité factuelle
- Particulièrement visible sur les noms propres et les données chiffrées
- Le throttling thermique aggrave ces comportements en fin de session longue

4.1.4 Support multilingue dégradé

Le français et les autres langues non-anglaises sont systématiquement moins bien supportés :

- Phi-3 Mini : -15 à 20 % de qualité en français vs anglais
- LLaMA 3.2 3B : biais anglophone marqué malgré 8 langues déclarées
- Seul Gemini Nano (via Gemini) offre un support multilingue robuste

4.1.5 Barrière de la reproductibilité : le cas Gemini Nano

À notre connaissance, **aucune étude académique publiée à ce jour ne fournit de benchmarks empiriques indépendants de Gemini Nano**. Cette absence n'est pas un oubli des chercheurs : elle reflète une limite structurelle.

Gemini Nano est un modèle entièrement propriétaire dont les poids ne sont pas distribués. L'accès passe obligatoirement par l'API ML Kit GenAI ou AICore, qui n'était pas publiquement disponible avant mi-2025. Par ailleurs, l'API n'expose pas les métriques brutes nécessaires à un benchmark rigoureux (prefill/decode en tokens/s, bande passante mémoire, utilisation NPU). Xu et al. [22], qui constituent la référence la plus complète sur les LLMs mobiles, ont délibérément exclu Gemini Nano de leur corpus pour cette raison et se limitent à llama.cpp et MLC-LLM : deux frameworks entièrement open source et reproductibles.

Notre PIR s'est heurté aux mêmes obstacles. La tentative de déploiement via ML Kit GenAI a révélé que le SDK n'est pas référencé sur Maven Central public, et les modèles "via AICore" se sont avérés désactivés sur le Galaxy S26 Ultra testé, bien que l'appareil figure sur la liste des appareils certifiés AICore par Google : la fonctionnalité Gemini multi-app associée est, à la date des tests, en déploiement bêta restreint à la Corée du Sud et aux États-Unis (Samsung, page d'assistance officielle TSG10010466), ce qui explique son indisponibilité depuis un appareil utilisé hors de ces deux marchés, indépendamment de toute limite matérielle. Le benchmark Google on-device a finalement été réalisé via LiteRT (AI Edge Gallery), qui constitue la solution Google on-device la plus proche accessible sans compte développeur ni matériel certifié.

Ce constat illustre une tension fondamentale dans le domaine : les solutions propriétaires les plus performantes (Gemini Nano, Apple Intelligence) sont précisément celles qui échappent à l'évaluation scientifique indépendante. La recherche reproductible sur les LLMs embarqués reste donc, en 2025-2026, quasi-exclusivement fondée sur les frameworks open source.

4.1.6 Les annonces d'accélération NPU face aux mesures de terrain

L'état de l'art (chapitre 1) rapporte les performances NPU telles qu'annoncées par le secteur : « les frameworks exploitant le NPU (AICore, LiteRT) peuvent offrir 2-4× le débit CPU à puissance équivalente » (section NPU vs CPU), et plus largement, « Apple Neural Engine, Snapdragon NPU (Hexagon), et les NPU Google Tensor montrent des performances 5-10× supérieures au CPU à consommation équivalente » (section 7.2). Le NPU est également présenté comme la voie d'accès aux « performances maximales » sur Gemini Nano via AICore (section 4.2).

Ces annonces n'ont pas résisté à l'épreuve du terrain sur le seul appareil flagship de notre corpus. Sur le Galaxy S26 Ultra (Snapdragon 8 Elite, 3 nm), l'appareil le plus susceptible de bénéficier de ces accélérations, l'application officielle Google exposant le chemin d'inférence on-device (AI Edge Gallery, LiteRT) a mesuré ~11-12 tok/s (estimés à partir de la latence totale, chapitre 2, section 3.4), soit environ 4× plus lent en débit que llama.cpp exécuté en CPU pur sur le même appareil ($46,68 \pm 14,40$ tok/s), et environ 2 à 3× plus lent en latence totale sur la même question (6,0 s contre 2,0 à 3,1 s). Le chemin le plus directement associé au NPU (Gemini Nano via AICore) s'est quant à lui révélé totalement indisponible, bien que l'appareil soit officiellement certifié AICore : la fonctionnalité Gemini multi-app qui l'exploite est, à la date des tests, restreinte par Google

à un déploiement bêta limité à la Corée du Sud et aux États-Unis (section 4.1.5), une restriction géographique indépendante des capacités matérielles du Snapdragon 8 Elite.

Annonce (chapitre 1)	Source	Mesure obtenue sur S26 Ultra (chapitre 2, §3.4)	Écart
NPU : 2-4× le débit CPU	\$NPU vs CPU	LiteRT (chemin officiel Google) : ~11-12 tok/s vs llama.cpp CPU : 46,68 tok/s	Inversé : CPU open source ~4× plus rapide en débit et 2 à 3× en latence (modèles différents : 2B contre 1B)
NPU : 5-10× supérieur au CPU	\$7.2	idem	Inversé
AICore = « performances maximales » sur flagship	\$4.2	AICore indisponible en pratique (déploiement bêta Google restreint à la Corée du Sud/États-Unis, cf. §4.1.5)	Non vérifiable depuis la Tunisie : restriction géographique du déploiement, non une limite matérielle
Gemini Nano-2 : 72 % GSM8K / 79,6 % MMLU	tableau comparatif \$3.3	« estimation basée sur les benchmarks Google », non reproduite indépendamment	Confirmé par notre propre incapacité à faire tourner le modèle

Tableau 4.3 : Annonces marketing NPU confrontées aux mesures de terrain (Galaxy S26 Ultra)

Une réserve méthodologique s'impose : ce résultat ne prouve pas que le NPU Hexagon du Snapdragon 8 Elite soit intrinsèquement plus lent qu'un CPU : les 11-12 tok/s mesurés proviennent de LiteRT, un chemin logiciel dont notre propre interprétation (chapitre 2, section 3.4) attribue la lenteur en partie à la taille du modèle (Gemma 4 E2B, 2B paramètres, contre 1B pour la comparaison llama.cpp ; de plus, ce débit est une estimation calculée sur la latence totale, soit ~70 tokens en 6,0 s, prefill et démarrage inclus, alors que celui de llama.cpp est un débit de decode mesuré : la comparaison n'est donc pas strictement équivalente) et à l'absence d'accélération NPU effective sur ce chemin précis, plutôt qu'à une limite matérielle du NPU lui-même. Ce que nos mesures démontrent, plus précisément, c'est que **l'outil officiel accessible à un développeur ou un utilisateur grand public** (celui que l'annonce marketing met en avant) ne délivre pas, dans les faits, la performance promise sur l'appareil et le canal qu'il expose. Que la cause soit un défaut de LiteRT, une NPU sous-exploitée, ou une combinaison des deux, la conclusion pratique reste la même : le discours commercial sur l'accélération NPU des solutions propriétaires ne s'est pas traduit, dans ce PIR, par un avantage mesurable face à un framework CPU open source pourtant réputé moins optimisé pour le matériel mobile.

4.1.7 Discussion : enjeux commerciaux et accessibilité de l'IA embarquée

Les sections 4.1.1 à 4.1.6 rapportent des limites mesurées. La présente section a un statut différent : elle discute, à partir de ces mesures, des enjeux d'accès à l'IA embarquée. Les

interprétations proposées relèvent de l'hypothèse et non du résultat, faute de données sur les décisions des constructeurs.

Nos mesures montrent d'abord que des appareils écartés des fonctionnalités IA officielles exécutent un LLM local via llama.cpp. Le Galaxy A71 (Snapdragon 730, lancé en 2019, sans mises à jour Samsung depuis 2023 et exclu de « Galaxy AI ») atteint $11,40 \pm 0,29$ tok/s en decode sous Termux natif (chapitre 2, tableau 2.8), et le Galaxy A16 5G, vendu environ 200 euros, $14,00 \pm 0,42$ tok/s, soit 3,9 à 7,8 tok/s en usage interactif avec le prototype (chapitre 3, tableau 3.5). À l'inverse, sur le Galaxy S26 Ultra, pourtant certifié AICore, le chemin Gemini Nano est resté indisponible en raison d'une restriction géographique du déploiement (section 4.1.5) et non d'une limite matérielle. Au vu de ces observations, l'éligibilité officielle aux fonctions IA semble en partie indépendante de la capacité matérielle effective ; nos données ne permettent toutefois pas d'établir pourquoi (choix commercial, contrainte de support logiciel ou garantie d'expérience utilisateur).

Cette situation pèse aussi sur la recherche. Apple Intelligence, Gemini Nano / AICore et Galaxy AI forment des écosystèmes distincts et non interopérables, et l'impossibilité d'obtenir des métriques brutes sur Gemini Nano (section 4.1.5) a empêché toute comparaison directe avec une solution propriétaire. De même, le label « AI phone », que la GSMA projette à 750 millions d'appareils en circulation d'ici 2028 [2], recouvre des définitions hétérogènes : nos mesures ne permettent pas de trancher entre traitement local réel et délégation au cloud, mais elles fournissent un étalon concret à opposer à ce label, celui d'un appareil d'entrée de gamme qui produit une inférence locale sans connexion réseau.

Enfin, l'usage de llama.cpp sur CPU ARM64 a permis de mener les mêmes mesures sur l'ensemble du corpus de six appareils, sans dépendre des fonctions propriétaires de chaque constructeur, ce qui rend ces résultats reproductibles. Cette portée est limitée au corpus testé et à des modèles d'environ un milliard de paramètres. Extrapoler à l'accessibilité de l'IA embarquée sur les marchés émergents, ou à un enjeu de souveraineté technologique, dépasserait ce que les mesures établissent ; cela reste une piste de discussion et de travaux futurs.

4.2 Pertinence pour des systèmes complexes (agents, MCP)

4.2.1 LLMs embarqués comme agents autonomes

L'utilisation de modèles on-device dans des architectures agentiques (type ReAct, Tool-use, MCP) est **théoriquement possible mais pratiquement limitée** :

Ce qui fonctionne :

- Agents simples avec 2-3 outils fixes (calculatrice, horloge, calendrier)
- Classification et routage de tâches vers le bon outil

- Extraction d'entités et structuration de données
- Agents "single-step" sans chaîne de pensée longue

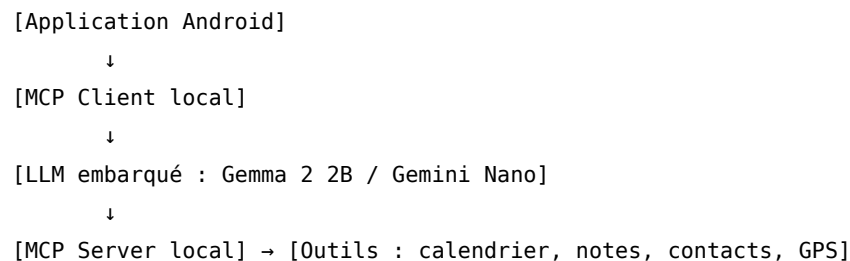
Ce qui ne fonctionne pas bien :

- ReAct (Reasoning + Acting) avec chaînes >3 étapes : le modèle perd le fil
- Tool-calling fiable : les modèles <4B génèrent des appels malformés fréquemment
- Planning long horizon : dégradation rapide avec la profondeur de la chaîne
- Self-correction : les petits modèles ont du mal à détecter leurs propres erreurs

4.2.2 Intégration avec le protocole MCP (Model Context Protocol)

Le MCP (Anthropic, 2024) définit un protocole standardisé pour connecter des LLMs à des outils externes. Son intégration avec des modèles embarqués présente des défis spécifiques :

Architecture MCP on-device envisageable :

**Problèmes identifiés :**

1. **Format JSON strict** : les modèles <4B génèrent du JSON malformé dans ~15-30 % des cas (dépend du prompt engineering)
2. **Latence cumulée** : chaque appel d'outil ajoute 1-3 secondes de délai. Une chaîne de 5 outils représente 15-20s de latence totale
3. **Gestion du contexte** : l'historique des appels d'outils consomme rapidement la fenêtre de contexte limitée

Solution pragmatique : utiliser le modèle embarqué uniquement pour le **routing** et la **classification**, et déléguer l'exécution complexe à un MCP server structuré avec des templates rigides :

Exemple : routage local + exécution structurée

```
def route_request(user_input: str, llm) -> str:
    # Le LLM embarqué classe l'intention (tâche simple)
    intent = classify_intent(user_input, llm) # "calendrier", "notes", "question"

    if intent == "calendrier":
        return calendar_tool.handle(user_input) # Logique déterministe
```

```
elif intent == "question":  
    return llm.generate(user_input)           # LLM pour les questions libres
```

4.2.3 Cas d’usage réalistes avec les LLMs embarqués

Cas d’usage	Faisabilité	Modèle recommandé	Notes
Chatbot FAQ local	Excellent	Gemma 2 2B / Gemini Nano	Cas d’usage principal
Résumé d’emails/articles	Très bon	Gemini Nano + ML Kit	ML Kit GenAI natif
Clavier intelligent	Bon	MobileLLM 1B	Faible latence requise
Classification de sentiment	Excellent	Gemma 2 2B	Très fiable
Extraction d’entités (NER)	Bon	Phi-3 Mini	Meilleur en anglais
Agent de planification	Limité	Phi-3 Mini uniquement	Max 3 étapes
Analyse de documents longs	Inadapté	Aucun	Contexte trop court
Raisonnement complexe	Inadapté	Aucun	Qualité insuffisante
Code generation complexe	Partiel	Phi-3 Mini	Simple seulement

Tableau 4.4 : Cas d’usage réalistes et faisabilité par modèle recommandé

4.3 Recommandations pour une architecture réaliste

4.3.1 Principe directeur : “Local by default, Cloud by exception”

L’architecture la plus pragmatique n’est ni entièrement locale ni entièrement cloud, mais **hybride avec un routeur intelligent** :

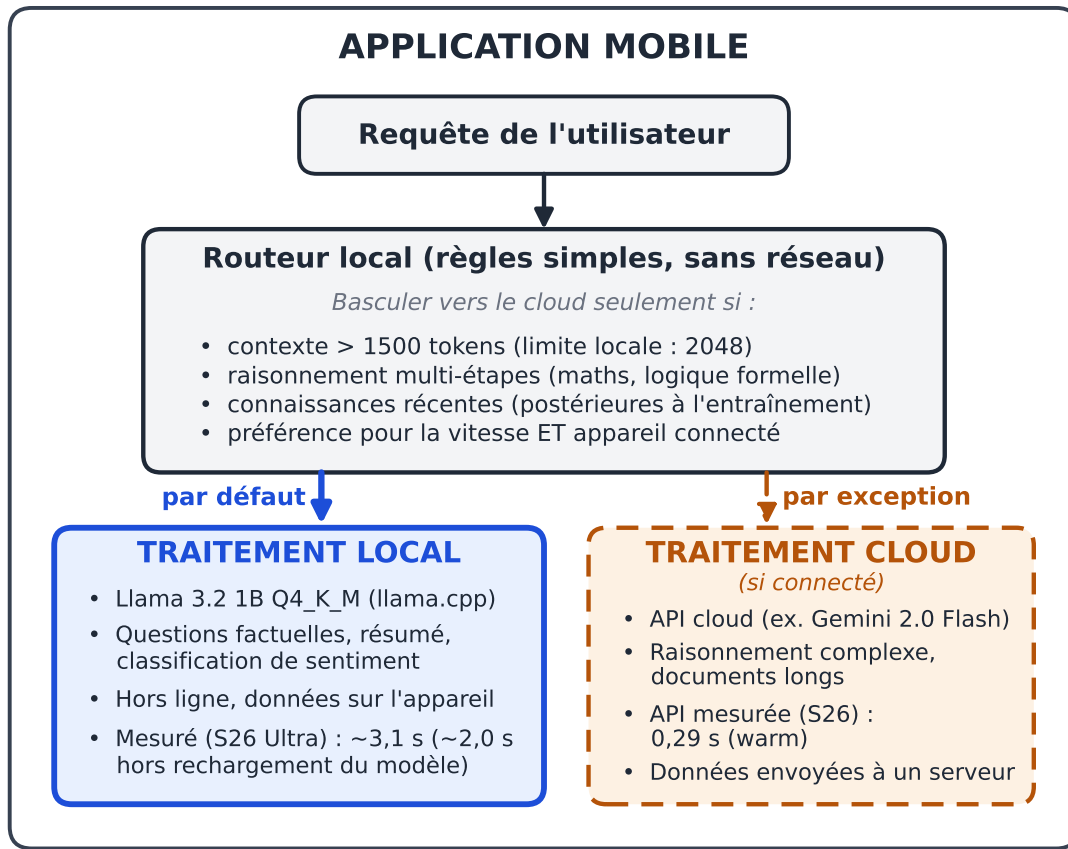


Figure 4.1 : Principe « Local by default, Cloud by exception » : routage d’une requête entre traitement local et cloud.

Critères de routage suggérés :

```
def should_use_cloud(request: str, context_length: int) -> bool:
    # Basculer vers le cloud si :
    return any([
        context_length > 1500,          # Contexte trop long pour le local
        contains_complex_reasoning(request), # Maths, logique formelle
        requires_recent_knowledge(request),  # Infos récentes (post-entraînement)
        user_prefers_speed and is_online(),  # Préférence vitesse + connecté
    ])
1)
```

Les deux critères les plus déterminants de ce routeur ne sont pas hypothétiques : ils reprennent directement des constats mesurés dans ce PIR. Le seuil `context_length > 1500` est calé sur la saturation de contexte observée empiriquement lors de la validation du prototype sur Galaxy S26 Ultra : l’erreur `request (2226 tokens) exceeds the available context size (2048 tokens)` est apparue après seulement ~13 échanges cumulés (chapitre 3, section 8.3 ; `journal_validation_prototype.md`). Le critère `contains_complex_`

reasoning(request) s’appuie quant à lui sur le taux d’échec mesuré du modèle LLaMA 3.2 1B sur les questions de raisonnement multi-étapes : seulement cinq réponses partielles (aucune pleinement correcte) sur trente-quatre tentatives au total (cinq questions × sept échantillons indépendants, moins une question non posée sur l’Infinix, chapitre 3, section 3.8.2), les échecs différant à chaque tentative plutôt que de suivre un motif stable : un signal que le problème n’est pas un défaut ponctuel du modèle mais une limite structurelle qui justifie un basculement systématique vers le cloud pour ce type de requête, plutôt qu’une simple tentative locale suivie d’un repli en cas d’échec. Ce constat a par ailleurs été confirmé de façon indépendante avec un second modèle et framework (Gemma3-1B-IT via Google AI Edge Gallery, en substitution de Gemma 2 2B, voir chapitre 3, section 3.8.2 et `journal_validation_prototype.md`), qui échoue lui aussi intégralement (0/5) sur ces mêmes questions de raisonnement.

4.3.2 Choix du framework selon le contexte

Contexte	Framework recommandé	Justification
App grand public, flagship récent	ML Kit GenAI (<i>sous réserve, voir note</i>)	API haut niveau, gestion du modèle déléguée à l’OS
App R&D, tous appareils	llama.cpp	Flexibilité maximale
Appareils Samsung/MediaTek	MLC-LLM	Seul à exploiter GPU Mali
Prototypage rapide	llama-cpp-python	Éco. Python, itération rapide
Production multiplateforme	MLC-LLM	Performance + couverture matérielle

Tableau 4.5 : Choix du framework d’inférence recommandé selon le contexte de déploiement

Note sur la ligne ML Kit GenAI : la littérature (et nos propres tableaux du chapitre 1) présente généralement ce choix comme justifié par la « performance NPU ». Nos propres mesures nuancent fortement cet argument (section 4.1.6) : sur le Galaxy S26 Ultra testé, AICore était indisponible en pratique (restriction géographique du déploiement bêta Google, cf. section 4.1.5), et le chemin LiteRT effectivement accessible mesurait ~11-12 tok/s, contre 46,68 tok/s pour llama.cpp en CPU pur sur le même appareil. ML Kit GenAI reste recommandé ici pour sa simplicité d’intégration côté développeur, et non pour un gain de performance que nos conditions de test n’ont pas permis de vérifier.

4.3.3 Recommandations pour les futurs travaux

1. **Explorer la quantification adaptative :** ajuster la précision par couche selon la sensibilité (certaines couches tolèrent INT2 sans perte notable). *Constat PIR :* notre propre test de raisonnement portant sur un calcul de RAM après quantification (chapitre 3, partie A.2, question 9) montre que le modèle 1B échoue déjà à appliquer

correctement une règle de quantification simple qu'on lui fournit dans le prompt : signe indirect que la marge de dégradation qualitative tolérable sur ce type de modèle est probablement plus étroite qu'annoncé, ce qui renforce l'intérêt d'une étude fine par couche avant tout déploiement en production.

2. **Implémenter le KV Cache partiel** : Apple montre qu'un KV Cache Sharing bien conçu réduit la mémoire de 37,5 %, applicable aux frameworks open source. *Constat PIR* : la fenêtre de 2 048 tokens s'est saturée après ~13 échanges lors de nos propres tests (chapitre 3, section 8.3) ; un KV Cache mieux géré ne repousserait pas la limite absolue de contexte, mais réduirait la pression mémoire qui, sur le S26 Ultra, se traduisait déjà par 4,9 Go de swap actif en usage léger (partie D, `journal_validation-prototype.md`).
3. **Systèmes multi-modèles** : un petit modèle (1B) pour le routage + un modèle moyen (3B) pour l'exécution : meilleure utilisation des ressources que d'un seul grand modèle. *Constat PIR* : cette recommandation est directement corroborée par nos résultats : le même modèle 1B qui classe correctement 12/12 textes de sentiment sur deux runs (chapitre 3, partie C) ne produit que cinq réponses partielles (aucune pleinement correcte) sur ses trente-quatre tentatives de raisonnement multi-étapes (chapitre 3, section 3.8.2), ce qui illustre concrètement pourquoi un seul modèle ne devrait pas être chargé de l'ensemble des tâches. Cette même limite de raisonnement a été retrouvée, de façon indépendante, sur un second modèle et framework d'inférence (Gemma3-1B-IT, Google AI Edge Gallery, chapitre 3, section 3.8.2), ce qui écarte l'hypothèse d'un défaut spécifique à l'implémentation retenue dans ce PIR.
4. **Évaluer PowerInfer-2 en production** [20] : la sparsité des activations est une piste prometteuse pour faire tourner des modèles 7B+ sur mobile. Xue et al. (2024) démontrent 29,2× d'accélération via décomposition en clusters de neurones (neuron cluster decomposition), permettant théoriquement l'exécution d'un modèle 47B sur smartphone. Ces résultats spectaculaires ne sont pas encore reproduits de manière indépendante, mais la direction algorithmique est prometteuse pour dépasser les limites actuelles de la bande passante mémoire. *Constat PIR* : le throttling thermique réel confirmé sur 2 des 6 appareils testés (Galaxy A16 : -19,1 %, Galaxy S26 Ultra : -17,3 %, chapitre 2, section 3.3) illustre concrètement la limite que ces approches de sparsité cherchent à repousser : moins de calcul actif par token signifie moins de chaleur dissipée, donc potentiellement moins de dégradation en session longue.
5. **Standard OS LLM** : contribuer ou adopter les standards émergents (type LLM as System Service) pour éviter la duplication des modèles entre applications. *Constat PIR* : ce standard répond directement à la duplication de modèle observée dans notre propre corpus : chaque application testée (prototype CLI, AI Edge Gallery) charge

indépendamment son propre modèle, sans partage, un gaspillage de RAM d'autant plus problématique sur les appareils d'entrée de gamme du corpus (5-6 Go de RAM totale sur l'A16/A71).

4.4 Conclusion critique

Les résultats obtenus aux chapitres 2 et 3 permettent de trancher le jugement de viabilité posé en introduction de ce mémoire non pas en bloc, mais cas d'usage par cas d'usage. C'est l'apport principal de la validation empirique menée en section 3.8.

Sur le raisonnement multi-étapes, le verdict est sans ambiguïté : sur les trente-quatre tentatives réalisées à travers les sept échantillons indépendants du corpus (trente-cinq questions prévues, la douzième n'ayant pu être posée sur l'Infinix, section 3.8.2), vingt-neuf réponses sont incorrectes et seulement cinq partielles ; aucune n'est pleinement correcte. Ce résultat n'est pas propre à LLaMA 3.2 1B ni à l'architecture du prototype : rejoué avec un second modèle et un second framework totalement indépendants (Gemma3-1B-IT via Google AI Edge Gallery), l'échec est total (0/5). Les LLMs embarqués de cette échelle de paramètres (~1 milliard) ne sont donc pas viables pour des tâches nécessitant un raisonnement enchaîné, quel que soit l'appareil ou le framework retenu : il s'agit d'une limite structurelle du modèle, non d'un défaut d'implémentation.

Sur les questions factuelles simples, en revanche, le bilan est positif mais imparfait : les scores s'échelonnent de 2/7 à 5/7 selon l'appareil et le run (S26 Ultra et A16 à 5/7, A71 et Infinix à 4/7, A26 à 3/7, A73 à 2/7), et une seule question sur sept reste parfaitement stable sur l'ensemble des sept échantillons (la capitale de la France). Cette instabilité résiduelle, y compris sur les questions les plus élémentaires du protocole, écarte l'idée d'une fiabilité totale, mais reste compatible avec un usage assisté où l'utilisateur peut repérer une réponse ponctuellement fausse, contrairement au raisonnement multi-étapes, où l'échec quasi systématique est plus difficile à détecter.

Le résumé de texte affiche un bilan intermédiaire favorable : cinq des huit runs testés obtiennent un verdict global « Bon », les trois autres restant « Partiel » plutôt qu'insuffisants ; le point le plus instable concerne la mention précise des optimisations matérielles (ARM NEON / NPU), rarement complète. La classification de sentiment, enfin, illustre le mieux la dépendance non seulement au matériel mais aussi au logiciel installé : le score varie de 1/6 (Infinix) à 6/6 (S26 Ultra, sur ses deux runs), une variance en grande partie attribuable non pas à une limite matérielle du SoC, mais à un incident logiciel reproductible identifié en section 3.8.3 (rejet du flag --no-conversation par certains builds de llama-cli, provoquant un rappel intégral de l'historique de conversation). Ce résultat nuance la lecture purement matérielle de la fragmentation Android évoquée ci-dessous : une partie au moins de la variabilité observée sur ce corpus relève de la reproductibilité logicielle plutôt que du silicium lui-même.

Plusieurs barrières structurelles restent donc identifiées :

- **Qualitatif** : le raisonnement multi-étapes reste hors de portée, sans exception observée sur six appareils et deux modèles/frameworks indépendants.
- **Matériel** : la fragmentation Android (GPU Mali non supporté par llama.cpp) complique le déploiement universel, bien qu'une partie de la variabilité mesurée sur ce corpus provienne en réalité de versions de build plutôt que du silicium lui-même.
- **Architectural** : l'absence de standard OS pour le service LLM force chaque application à dupliquer le modèle en mémoire.

Au global, les LLMS embarqués sur smartphone se révèlent donc viables, avec une fiabilité imparfaite mais assumable, pour des cas d'usage bien circonscrits (questions factuelles courtes, classification de sentiment, résumé de texte), et non viables pour le raisonnement multi-étapes complexe. L'architecture hybride, avec un modèle local pour les tâches courantes et le cloud pour les cas complexes, représente en conséquence le compromis le plus réaliste pour les 2-3 prochaines années, jusqu'à ce que les modèles embarqués franchissent le seuil qualitatif des 7-10B paramètres quantifiés en 2 bits.